

Disha Jadav

San Jose, CA | dishajadav12402@gmail.com | github.com/dishajadav12 | linkedin.com/in/disha-jadav-606484209 | +1 408-549-5820

EDUCATION

San Jose State University

Master of Science, Software Engineering, **GPA: 3.67 / 4.0**

Courses: Distributed Systems, Systems Software, Enterprise Software Platforms, Data Mining

California, USA

Aug 2025 - May 2027

A.D. Patel Institute of Technology

B.E. in Computer Engineering, **CGPA: 8.3 / 10**

Courses: Data Structures & Algorithms, Operating Systems, Computer Networks

Gujarat, India

Aug 2020 - May 2024

TECHNICAL SKILLS

Languages: Python, TypeScript, JavaScript, Java, Go, C++, SQL

Web & Frameworks: React, Next.js, Node.js, FastAPI, Spring Boot, GraphQL, gRPC, **REST APIs**, Kafka, Celery, Redux, WebSockets, HTML5, Tailwind CSS

Databases & Infrastructure: PostgreSQL, MongoDB, Redis, MySQL, Docker, Kubernetes, Terraform, CI/CD, Git,

Cloud & Observability: AWS (EC2, S3, ECS, Lambda), GCP, Prometheus, Loki, Jaeger, OpenTelemetry, Grafana

Core Concepts: Full-Stack Development, Distributed Systems, End-to-End Ownership, System Design, API Design, Concurrency, Data Structures & Algorithms

PROFESSIONAL EXPERIENCE

Founding Engineering Intern, Heritage Table

May 2026 - Aug 2026

- Sole engineer who built a consumer web and mobile product end-to-end across the full stack in **React**, **TypeScript**, and **Python** on a **Postgres** backend, owning the architecture, data model, APIs, and infrastructure, and shipping continuously to **50+ beta users** while iterating directly on their feedback.
- Designed a database-enforced authorization model that scopes every read and write to the correct user at the data layer, and built a print-on-demand **commerce pipeline** with **Stripe** checkout and webhook-driven order tracking that carries a user from design through payment to a physical product.
- Built **AI-powered features** with asynchronous processing that turn documents, images, and voice into structured content, and a real-time collaborative editing experience with live presence that lets multiple users edit the same content without overwriting each other.

Full Stack Developer, Neural Web Technologies

Apr 2024 - Jul 2025

- Designed and built a **real-time Node.js/GraphQL** platform serving thousands of concurrent users, engineering conflict detection, live chat over **WebSockets**, and recommendation logic, and driving measurable product gains including a **40%** reduction in scheduling time and a **15%** lift in weekly retention.
- Optimized **performance-critical** paths across a system sustaining **10,000+** concurrent users under live-event load: profiled hot paths, added **Redis** caching and query optimization, and reworked database access to cut API response time **30%** while keeping the service responsive at peak.
- Diagnosed and resolved a production incident through cross-layer **root-cause analysis**: when a cache-eviction change degraded a neighboring system, traced it through **monitoring**, fixed it with namespaced keys within hours, then hardened observability with a hit-rate alarm and regression tests so the failure could not recur silently.

PROJECTS

K8s Observability Signal Aggregator (Go, Python, OpenTelemetry, Docker, Kubernetes)

[GitHub](#)

- Built with a small team a distributed observability tool in **Go** and **Python** that queries and correlates Prometheus metrics, Loki logs, and Jaeger traces in parallel across **4 microservices** in **under 2 seconds**, surfacing the root cause of an incident with direct links back to the supporting signals.
- Designed a configurable **rule engine** for threshold-based signal filtering, validated against real failure modes including dropped connections, cascading errors, and slow responses, so operators can tune what counts as an anomaly without changing code.
- Added an **LLM** analysis layer that reads the correlated signals and produces a plain-English root-cause summary with references, turning raw metrics, logs, and traces into an explanation an engineer can act on quickly.

EnvDrift (Python, FastAPI, Kafka, Celery, PostgreSQL, Redis, Docker)

[GitHub](#)

- Independently designed and built a distributed **data pipeline** in **Python/FastAPI** using **Kafka** for job distribution and **Celery** worker pools for asynchronous parallel processing, exposed through a clean **REST API** and a single-command CLI that reports configuration drift across an entire environment in one pass.
- Eliminated duplicate alerts by adding **idempotency keys** and **Redis**-backed deduplication, resolving a concurrent-worker race condition where the same notification fired multiple times, and kept processing consistent as the worker pool scaled up under load.